ML Container Creator

One CLI command → a complete, deployable SageMaker BYOC project.

```
npm install -g @aws/ml-container-creator
ml-container-creator
```

`@aws/ml-container-creator` · v0.5.0 · Apache-2.0

> Speaker: "How many of you have written a Dockerfile for SageMaker from scratch? MCC eliminates that boilerplate entirely."

The Problem

Why BYOC Is Hard

Challenge	Impact
Boilerplate	Dockerfile + serve script + health check + deploy per model
Framework diversity	vLLM ≠ Triton ≠ Flask — different images, ports, configs
48 instance types	16 families — which fits your model?
Misconfigurations	Wrong CUDA, missing /ping, bad port → hours lost
4 deploy targets	Real-time, async, batch, HyperPod — different infra each
Secrets	HF tokens, NGC keys need Secrets Manager

> Speaker: Teams spend 2-5 days on container boilerplate before testing inference. "Who has deployed the same model to both real-time and async? How much code did you duplicate?"

What MCC Does

Code Generator, Not Runtime Framework

```
You select:                          MCC generates:
• Model                              • Dockerfile
• Serving backend      ──────►       • Serving code
• Deploy target                      • 18 do/ lifecycle scripts
• Instance type                      • Tests + IAM docs + README
```

Key principles:

- ✅ You own the output — modify, commit, maintain
- ✅ No runtime dependency — MCC is not in your container
- ✅ Opinionated defaults, full escape hatches

> Speaker: "Think create-react-app for ML containers. No lock-in, no runtime

Supported Architectures

4 Families — 15 Deployment Configs

Architecture	Backends	#	Use Case
HTTP	Flask, FastAPI	2	sklearn, XGBoost, TensorFlow
Transformers	vLLM, SGLang, TensorRT-LLM, LMI, DJL	5	LLMs
Triton	FIL, ONNX, TF, PyTorch, vLLM, TRT-LLM, Python	7	Multi-framework
Diffusors	vLLM-Omni	1	Image generation

```
ml-container-creator my-model \
  --deployment-config=transformers-vllm \
  --model-name=meta-llama/Llama-2-7b-chat-hf \
  --instance-type=ml.g5.2xlarge --skip-prompts
```

> Speaker: "15 tested combinations of Dockerfile + serve code + deploy scripts."

Live Demo

Zero to Deployed in 4 Commands

```
ml-container-creator my-llm \
  --deployment-config=transformers-vllm \
  --model-name=mistralai/Mistral-7B-Instruct-v0.2 \
  --instance-type=ml.g5.2xlarge

cd my-llm
./do/build        # Build Docker image
./do/run          # Run locally
./do/test         # Health check + inference
./do/push         # Push to ECR
./do/deploy       # Create SageMaker endpoint
```

18 do/ scripts: build · run · test · push · deploy · clean · status · logs · register · benchmark · adapter · add-ic · optimize · validate · config · export · ci · submit

MCP Servers

6 Bundled Servers — Intelligent Defaults

Server	Tool	Purpose
instance-sizer	`get_instance_recommendation`	Model size → instance type
region-picker	`get_regions`	Availability + latency
base-image-picker	`get_base_images`	Framework → optimal image
model-picker	`get_models`	HuggingFace / JumpStart / S3
hyperpod-cluster-picker	`get_hyperpod_clusters`	Available EKS clusters
endpoint-picker	`get_inference_endpoints`	Existing endpoints

Static mode — catalog data, no credentials

Smart mode — live AWS APIs for quotas & pricing

Configuration for CI/CD

8-Level Precedence Chain

```
1. CLI flags                (highest)
2. Environment variables
3. Config file
4. Preset files
5. MCP server responses
6. Bootstrap config
7. Schema defaults
8. Hardcoded defaults       (lowest)
```

```
export MCC_DEPLOYMENT_CONFIG=transformers-vllm
export MCC_MODEL_NAME=meta-llama/Llama-2-7b-chat-hf
export MCC_INSTANCE_TYPE=ml.g5.2xlarge
ml-container-creator my-model --skip-prompts
```


> Speaker: "Every parameter: CLI flag, env var, or config file. In CI, set env vars + --

Deployment Targets

Same Container — 4 Targets

Target	Description	Key Feature
Managed Inference	Real-time endpoints	Multi-IC, auto-scaling
Async Inference	S3-based async	SNS notifications, large payloads
Batch Transform	S3-to-S3 bulk	Cost-efficient dataset processing
HyperPod EKS	Kubernetes	GPU scheduling, heterogeneous clusters

Multi-IC + LoRA Adapters:

```
./do/deploy                                          # Base model
./do/add-ic --model-name=codellama/CodeLlama-7b-hf  # 2nd model
./do/adapter add --name=my-lora --path=s3://...      # Hot-swap adapter
```


> Speaker: "Same Dockerfile across all 4 targets. Multi-IC = multiple models on

What You Get

Generated Project Structure

```
my-model/
├── Dockerfile              # Multi-stage, SageMaker-optimized
├── requirements.txt
├── code/
│   ├── model_handler.py    # Load + inference logic
│   ├── serve.py            # Server (Flask/FastAPI)
│   └── serving.properties  # LMI/DJL config
├── do/                     # 18 lifecycle scripts
│   ├── build, run, test, push, deploy, clean
│   ├── adapter, add-ic, benchmark, optimize
│   ├── register, status, logs, validate
│   └── lib/                # Shared helpers
├── hyperpod/               # K8s manifests (if EKS)
├── test/                   # Unit tests
├── IAM_PERMISSIONS.md      # Required policies
└── README.md               # Next steps
```

Registry & Validation

3 Catalogs — Validated Combinations

Catalog	Entries	Contains
model-servers	13	Versions, base images, CUDA
models	Popular LLMs + diffusors	Families, sizes, instances
instances	48 / 16 families	GPU, memory, cost tier

Validation Levels

	Level	Meaning
✅	tested	CI-validated (vLLM, TRT-LLM, LMI)
🟡	community	User-reported (DJL)
🔬	experimental	Generated, not CI-tested (SGLang, Triton)

Deployment Registry

Capture · Replay · Export · Search

```
./do/register                                  # Save after deploy
ml-container-creator registry list             # All deployments
ml-container-creator registry replay prod  # Redeploy exact config
ml-container-creator registry export > d.json  # Share
ml-container-creator registry import d.json    # Import
ml-container-creator registry search --model="Mistral"
```

Captures: config, model, instance, region, timestamp, endpoint name, full params

> Speaker: "Six months later, someone needs to redeploy. Registry has the exact config." Complements IaC — captures MCC-specific parameters.

What It's Not

Honest Positioning

MCC Is	MCC Is Not
Code generator	Managed service
SageMaker-compatible starter code	Production-ready without review
Head start on BYOC	Replacement for JumpStart
Open-source (Apache-2.0)	AWS service with SLA

Use alternatives when:

- Model is in JumpStart catalog → use JumpStart
- Want Python SDK approach → use Inference Toolkit
- Unique requirements outside 15 configs → build from scratch

Getting Started

Prerequisites & Install

Tool	Version	Purpose
Node.js	24+	CLI runtime
Docker	20+	Container builds
AWS CLI	2+	AWS management
Python	3.8+	Serving code

```
npm install -g @aws/ml-container-creator
ml-container-creator bootstrap    # One-time AWS setup
ml-container-creator              # Generate project
```

📖 Docs · 📋 Examples · 🐛 Issues

> Speaker: "Bootstrap creates IAM role, ECR repo, optional SZ + CI harness. ~2

Roadmap

✅ Shipped · 📐 Designed · 📋 Planned

✅ Shipped (v0.3.0 → v0.5.0)

Feature	Feature
Async/Batch/HyperPod targets	Secrets Manager (HF_TOKEN, NGC_API_KEY)
Schema-driven validation	Instance right-sizing MCP
Training Plan reservations	CUDA auto-resolution
CLI config + --skip-prompts	Yeoman removal (standalone)
CI integration harness	Bootstrap shared infra
Deployment registry	Multi-IC endpoints
LoRA adapter lifecycle	Post-deploy guidance
Instance quota & availability	Benchmarking (`do/benchmark`)

Thank You + Q&A

Try It Today

```
npx @aws/ml-container-creator
```

5 minutes to generate, build, and test locally — no AWS credentials needed.

GitHub: awslabs/ml-container-creator
Docs: awslabs.github.io/ml-container-creator

> Speaker: Reiterate 5-minute trial. Common Q&A: security review, multi-region, cost estimation, team onboarding.

Appendix A: Why a Code Generator

Approach	Pros	Cons
MCC	Full control, no lock-in, auditable	Must maintain generated code
CLI wrapper	Simple	Black box, limited customization
SDK (Inference Toolkit)	Pythonic	Still need Dockerfile + scripts
Managed (JumpStart)	Zero container work	Limited models, no customization
IaC only	Infra as code	No serving code generation

> Speaker: "MCC is complementary to IaC. It generates the application layer; CDK/Terraform handles infrastructure."

Appendix B: Model Servers & Base Images

Server	Version	Base Image	CUDA	Level
vLLM	v0.10.1	vllm/vllm-openai:v0.10.1	12.4	✅
vLLM	v0.9.1	vllm/vllm-openai:v0.9.1	12.1	✅
SGLang	v0.5.4.post1	lmsysorg/sglang:v0.5.4.post1-cu121	12.1	🔬
TRT-LLM	1.2.0rc8	nvcr.io/.../tensorrt-llm:1.2.0rc8	12.4	✅
TRT-LLM	1.1.0	nvcr.io/.../tensorrt-llm:1.1.0	12.1	✅
LMI	0.32.0	763104351884...djl-inference:0.32.0-lmi14	12.6	✅
LMI	0.31.0	763104351884...djl-inference:0.31.0-lmi13	12.4	✅
DJL	0.36.0	djl-serving:0.36.0-pytorch-gpu	12.6	🟡
vLLM-Omni	v0.16.0	vllm/vllm-omni:v0.16.0	12.4	🔬
Triton	24.08	nvcr.io/.../tritonserver:24.08-py3	12.5	🔬

Appendix C: Parameter Reference

Parameter	CLI Flag	Env Var	Default
Deployment config	`--deployment-config`	`MCC_DEPLOYMENT_CONFIG`	—
Model name	`--model-name`	`MCC_MODEL_NAME`	—
Instance type	`--instance-type`	`MCC_INSTANCE_TYPE`	—
Region	`--region`	`MCC_REGION`	us-east-1
Deploy target	`--deploy-target`	`MCC_DEPLOY_TARGET`	sagemaker
Skip prompts	`--skip-prompts`	`MCC_SKIP_PROMPTS`	false

IC Parameters: cpuCount (0.25–768) · memorySize (128–3145728 MB) · gpuCount (0–8) · copyCount (0–100) · modelWeight (0–1)

> Speaker: "Every parameter follows the 8-level precedence chain. IC params enable fine-grained multi-model resource allocation."

Appendix D: Instance Types

GPU (40 types / 13 families)

Family	GPU	VRAM	Arch	Tier	Sizes
g4dn	T4	16 GB	Turing	$	6
g5	A10G	24 GB	Ampere	$$	8
g6	L4	24 GB	Ada	$$	3
g6e	L40S	48 GB	Ada	$$	7
g7e	RTX PRO 6000	96 GB	Blackwell	$$	6
p3	V100	16 GB	Volta	$$$	3
p4d	A100	40 GB	Ampere	$$$	1
p5	H100	80 GB	Hopper	$$$	1
p5e/p5en	H200	141 GB	Hopper	$$$	2